Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 2.5

Obtained Marks: _____

Artificial Neural Networks

Lab Task # 01

Last date of Submission: October 13th, 2026

Submitted To: Muhammad Hasaan Mujtaba

Student Name: Muhammad Hassaan

Reg. Number: 24108249

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Q : Take any 2 different dataset on regression and implement linear regression on both of them and make user interface through gradio as well as through flask

Note: all students will use different datasets

Dataset: Student Performance

jupyter ANN TASK 1 Last Checkpoint: 3 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

```
[ ]: %pip install pandas numpy scikit-learn joblib gradio flask
```

```
[19]: import pandas as pd
      import numpy as np
      import joblib

      from sklearn.model_selection import train_test_split
      from sklearn.linear_model import LinearRegression
      from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

      print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
[16]: df = pd.read_csv(r"C:\Users\92335\Downloads\Student_Performance.csv")

      print("Dataset loaded successfully!")
      print("Dataset shape:", df.shape)

      df.head()
```

```
Dataset loaded successfully!
Dataset shape: (10000, 6)
```

[16]:

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced	Performance Index
0	7	99	Yes	9	1	91.0
1	4	82	No	4	2	65.0
2	8	51	Yes	7	2	45.0
3	5	52	Yes	5	2	36.0
4	7	75	No	8	5	66.0

```
[18]: print("Column names :")
      print(df.columns.tolist())
```

```
Column names :
['Hours Studied', 'Previous Scores', 'Extracurricular Activities', 'Sleep Hours', 'Sample Question Papers Practiced', 'Performance Index']
```

```
[21]: print("Dataset information :")
      df.info()
```

```
Dataset information :
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
 #   Column                            Non-Null Count  Dtype
---  ------                            --------------  -----
 0   Hours Studied                     10000 non-null  int64
 1   Previous Scores                   10000 non-null  int64
 2   Extracurricular Activities        10000 non-null  str
 3   Sleep Hours                       10000 non-null  int64
 4   Sample Question Papers Practiced  10000 non-null  int64
 5   Performance Index                 10000 non-null  float64
dtypes: float64(1), int64(4), str(1)
memory usage: 468.9 KB
```

[22]:
```
print("Missing values :")
print(df.isnull().sum())
```

```
Missing values :
Hours Studied                       0
Previous Scores                     0
Extracurricular Activities          0
Sleep Hours                         0
Sample Question Papers Practiced    0
Performance Index                   0
dtype: int64
```

[23]:
```
print("Duplicate records :")
print(df.duplicated().sum())
```

```
Duplicate records :
127
```

[24]:
```
print("Statistical summary:")
display(df.describe())
```

Statistical summary:

	Hours Studied	Previous Scores	Sleep Hours	Sample Question Papers Practiced	Performance Index
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	4.992900	69.445700	6.530600	4.583300	55.224800
std	2.589309	17.343152	1.695863	2.867348	19.212558
min	1.000000	40.000000	4.000000	0.000000	10.000000
25%	3.000000	54.000000	5.000000	2.000000	40.000000
50%	5.000000	69.000000	7.000000	5.000000	55.000000
75%	7.000000	85.000000	8.000000	7.000000	71.000000
max	9.000000	99.000000	9.000000	9.000000	100.000000

[25]:
```
df = df.drop_duplicates().copy()

df = df.dropna().copy()

df["Extracurricular Activities"] = (
    df["Extracurricular Activities"]
    .map({"Yes": 1, "No": 0})
)

df = df.dropna().copy()

print("Data cleaned successfully!")
print("Remaining records:", len(df))

display(df.head())
```

```
Data cleaned successfully!
Remaining records: 9873
```

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced	Performance Index
0	7	99	1	9	1	91.0
1	4	82	0	4	2	65.0
2	8	51	1	7	2	45.0
3	5	52	1	5	2	36.0
4	7	75	0	8	5	66.0

[27]:
```
features = ["Hours Studied","Previous Scores","Extracurricular Activities","Sleep Hours","Sample Question Papers Practiced"]

X = df[features]
y = df["Performance Index"]

print("Input features (X):")
display(X.head())

print("Target variable (y):")
display(y.head())
```

```
Input features (X):
```

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced
0	7	99	1	9	1
1	4	82	0	4	2
2	8	51	1	7	2
3	5	52	1	5	2
4	7	75	0	8	5

```
Target variable (y):
0    91.0
1    65.0
2    45.0
3    36.0
4    66.0
Name: Performance Index, dtype: float64
```

[28]:
```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20,random_state=42)

print("Training records:", len(X_train))
print("Testing records:", len(X_test))
```

```
Training records: 7898
Testing records: 1975
```

[29]:
```
model = LinearRegression()

model.fit(X_train, y_train)

print("Linear Regression model trained successfully!")
```

```
Linear Regression model trained successfully!
```

[31]:
```
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MODEL EVALUATION")

print("\nMAE:", round(mae, 3))
print("MSE:", round(mse, 3))
print("RMSE:", round(rmse, 3))
print("R2 Score:", round(r2, 4))
```

```
MODEL EVALUATION

MAE: 1.647
MSE: 4.306
RMSE: 2.075
R2 Score: 0.9884
```

[32]:
```
coefficients = pd.DataFrame({"Feature": features,"Coefficient": model.coef_})

display(coefficients)

print("Intercept:", model.intercept_)
```

	Feature	Coefficient
0	Hours Studied	2.851022
1	Previous Scores	1.018430
2	Extracurricular Activities	0.573823
3	Sleep Hours	0.472073
4	Sample Question Papers Practiced	0.188704

```
Intercept: -33.981324496440706
```

[33]:
```
new_student = pd.DataFrame([{"Hours Studied": 6,"Previous Scores": 88,"Extracurricular Activities": 1,"Sleep Hours": 7,"Sample Question Papers Practi

predicted_score = model.predict(new_student)[0]

print("Predicted Performance Index:", round(predicted_score, 2))
```

```
Predicted Performance Index: 69.23
```

[34]:
```
joblib.dump({"model": model,"features": features},"student_model.pkl")

print("Model saved successfully!")
```

```
Model saved successfully!
```

[2]:
```
import gradio as gr

def predict_performance(
    hours_studied,
    previous_scores,
    extracurricular,
    sleep_hours,
    practice_papers
):
    student = pd.DataFrame([{
        "Hours Studied": hours_studied,
        "Previous Scores": previous_scores,
        "Extracurricular Activities": (
            1 if extracurricular == "Yes" else 0
        ),
        "Sleep Hours": sleep_hours,
        "Sample Question Papers Practiced": practice_papers
    }], columns=features)

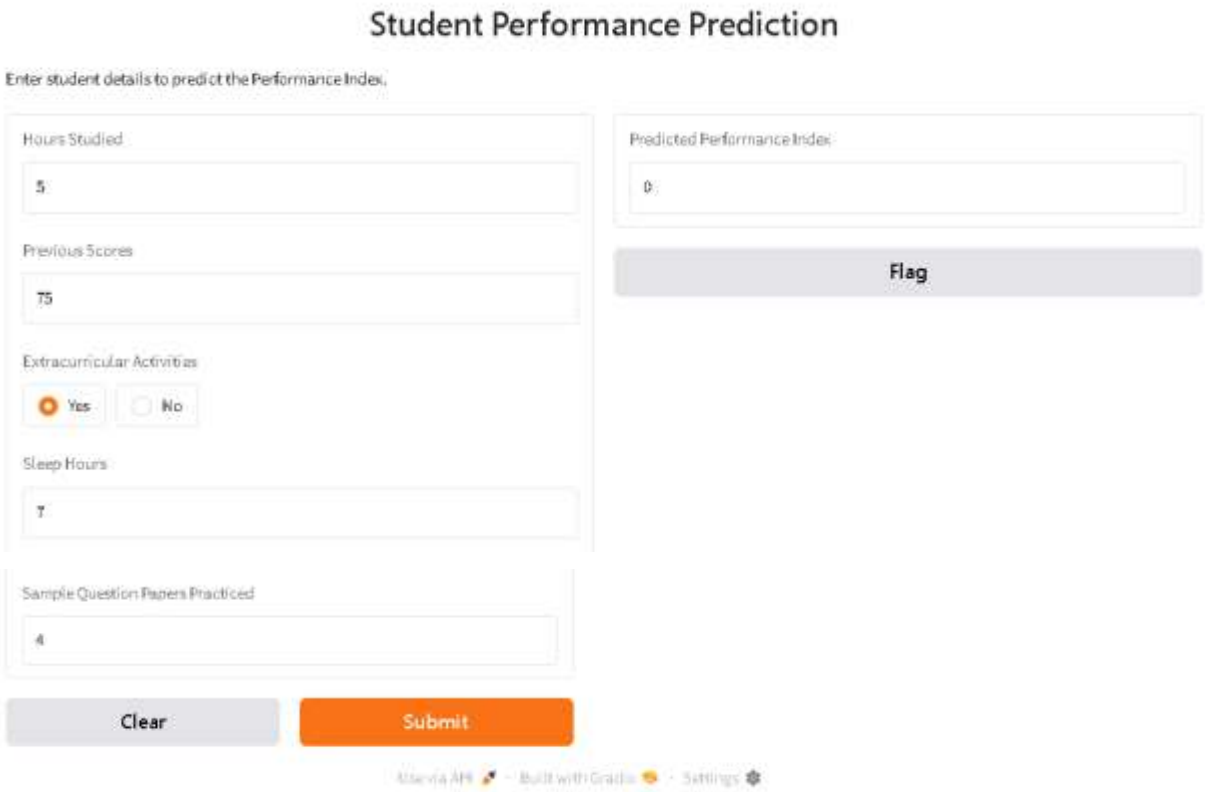
    prediction = model.predict(student)[0]

    return round(float(prediction), 2)
```

```
demo = gr.Interface(
    fn=predict_performance,
    inputs=[
        gr.Number(label="Hours Studied", value=5),
        gr.Number(label="Previous Scores", value=75),
        gr.Radio(
            choices=["Yes", "No"],
            label="Extracurricular Activities",
            value="Yes"
        ),
        gr.Number(label="Sleep Hours", value=7),
        gr.Number(
            label="Sample Question Papers Practiced",
            value=4
        )
    ],
    outputs=gr.Number(label="Predicted Performance Index"),
    title="Student Performance Prediction",
    description="Enter student details to predict the Performance Index."
)

demo.launch()
```

```
* Running on local URL:  http://127.0.0.1:7862
* To create a public link, set `share=True` in `launch()`.
```



[36]:
```
from pathlib import Path

Path("templates").mkdir(exist_ok=True)

print("Templates folder created!")
```

```
Templates folder created!
```

[37]:
```
html_code = """
<!DOCTYPE html>
<html>
<head>
    <title>Student Performance Prediction</title>
    <style>
        body {
            font-family: Arial;
            background: #f2f4f8;
            padding: 30px;
        }
        .container {
            max-width: 500px;
            margin: auto;
            background: white;
            padding: 25px;
            border-radius: 12px;
        }
```

```
        label {
            display: block;
            margin-top: 15px;
        }
        input, select, button {
            width: 100%;
            padding: 10px;
            margin-top: 6px;
            box-sizing: border-box;
        }
        button {
            margin-top: 20px;
            background: #2457c5;
            color: white;
            border: none;
            cursor: pointer;
        }
        .result {
            margin-top: 20px;
            background: #e7f7e9;
            padding: 12px;
            text-align: center;
        }
    </style>
</head>
<body>
    <div class="container">
        <h2>Student Performance Prediction</h2>

        <form method="POST">
            <label>Hours Studied</label>
            <input type="number" name="hours_studied"
                   min="0" step="any" required>

            <label>Previous Scores</label>
            <input type="number" name="previous_scores"
                   min="0" step="any" required>

            <label>Extracurricular Activities</label>
            <select name="extracurricular">
                <option value="Yes">Yes</option>
                <option value="No">No</option>
            </select>

            <label>Sleep Hours</label>
            <input type="number" name="sleep_hours"
                   min="0" step="any" required>
```

```
            <label>Sample Question Papers Practiced</label>
            <input type="number" name="practice_papers"
                   min="0" step="1" required>

            <button type="submit">Predict Performance</button>
        </form>

        {% if prediction is not none %}
        <div class="result">
            <h3>Predicted Performance Index</h3>
            <h2>{{ prediction }}</h2>
        </div>
        {% endif %}

        {% if error %}
        <p>{{ error }}</p>
        {% endif %}
    </div>
</body>
</html>
"""

Path("templates/index.html").write_text(html_code, encoding="utf-8")

print("HTML template created successfully!")
```

```
print("HTML template created successfully!")
```

```
HTML template created successfully!
```

[38]:

```
from flask import Flask, render_template, request

app = Flask("student_performance_app")

@app.route("/", methods=["GET", "POST"])
def home():
    prediction = None
    error = None

    if request.method == "POST":
        try:
            hours_studied = float(request.form["hours_studied"])
            previous_scores = float(request.form["previous_scores"])
            extracurricular = request.form["extracurricular"]
            sleep_hours = float(request.form["sleep_hours"])
            practice_papers = float(request.form["practice_papers"])
```

```
            ) < 0:
                raise ValueError

            if extracurricular not in ["Yes", "No"]:
                raise ValueError

            student = pd.DataFrame([{
                "Hours Studied": hours_studied,
                "Previous Scores": previous_scores,
                "Extracurricular Activities": (
                    1 if extracurricular == "Yes" else 0
                ),
                "Sleep Hours": sleep_hours,
                "Sample Question Papers Practiced": practice_papers
            }], columns=features)

            prediction = round(
                float(model.predict(student)[0]), 2
            )

        except (ValueError, KeyError):
            error = "Please enter valid, non-negative values."

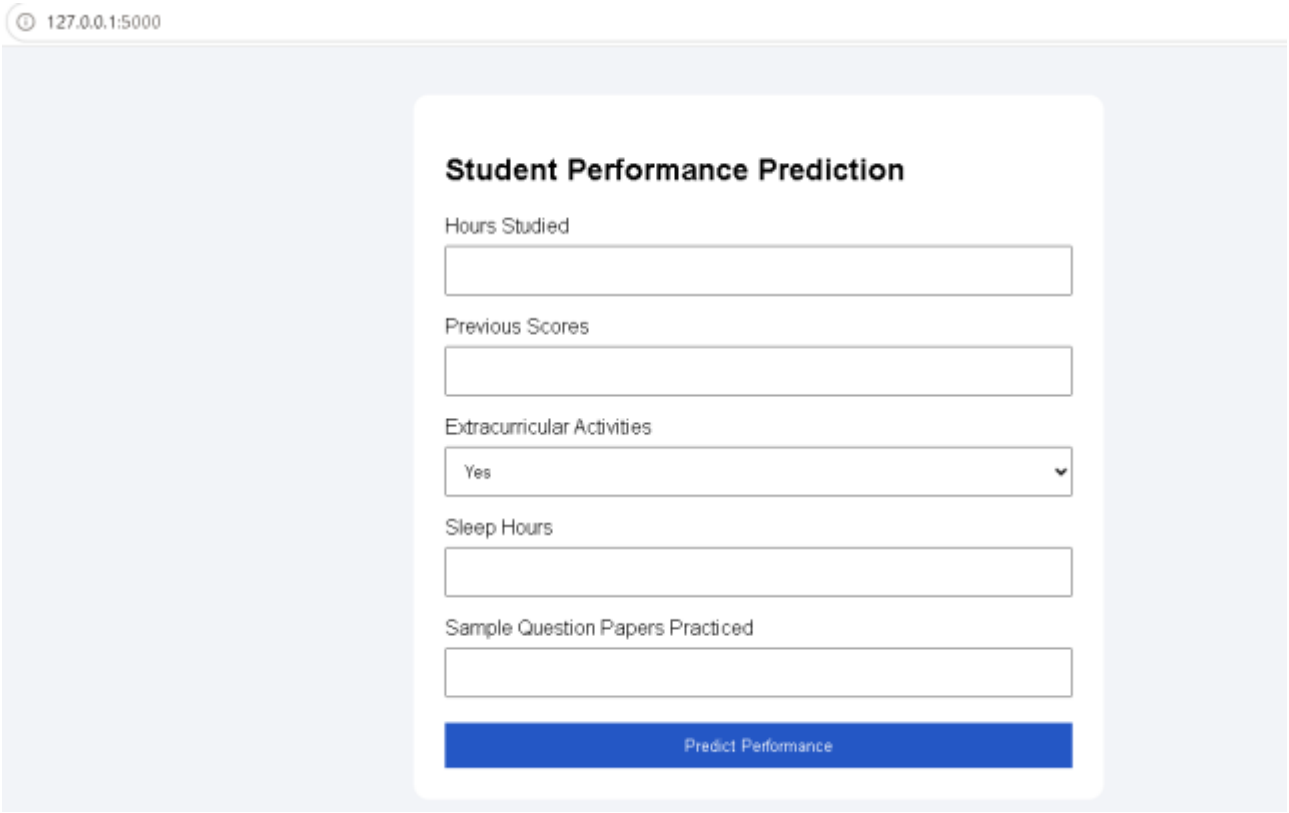
    return render_template(
        "index.html",
        prediction=prediction,
        error=error
    )

print("Flask app is ready!")
```

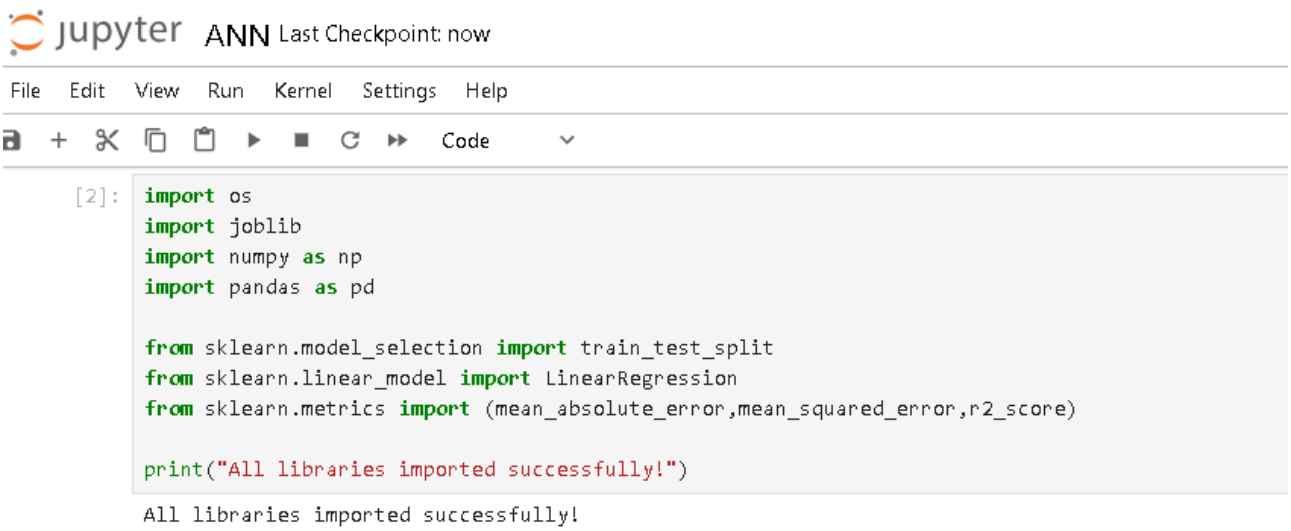
```
Flask app is ready!
```

```
app.run(host="127.0.0.1", port=5000, debug=False, use_reloader=False)
```

```
 * Serving Flask app 'student_performance_app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [12/Oct/2026 23:48:32] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Oct/2026 23:48:33] "GET /favicon.ico HTTP/1.1" 404 -
```



Dataset: Auto MPG



[4]:
```
df = pd.read_csv(r"C:\Users\92335\Downloads\auto-mpg.csv")

print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)

display(df.head())
```

```
Dataset loaded successfully!
Dataset shape: (398, 9)
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino

[5]:
```
print("Column names:")
print(df.columns.tolist())

print("\nDataset information:")
df.info()

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())

display(df.describe())
```

```
Column names:
['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model year', 'origin', 'car name']

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64
 2   displacement  398 non-null    float64
 3   horsepower    398 non-null    str
 4   weight        398 non-null    int64
 5   acceleration  398 non-null    float64
 6   model year    398 non-null    int64
 7   origin        398 non-null    int64
 8   car name      398 non-null    str
dtypes: float64(3), int64(4), str(2)
memory usage: 28.1 KB

Missing values:
mpg             0
cylinders       0
displacement    0
horsepower      0
weight          0
acceleration    0
model year      0
origin          0
car name        0
dtype: int64

Duplicate rows: 0
```

```
Duplicate rows: 0
```

	mpg	cylinders	displacement	weight	acceleration	model year	origin
count	398.000000	398.000000	398.000000	398.000000	398.000000	398.000000	398.000000
mean	23.514573	5.454774	193.425879	2970.424623	15.568090	76.010050	1.572864
std	7.815984	1.701004	104.269838	846.841774	2.757689	3.697627	0.802055
min	9.000000	3.000000	68.000000	1613.000000	8.000000	70.000000	1.000000
25%	17.500000	4.000000	104.250000	2223.750000	13.825000	73.000000	1.000000
50%	23.000000	4.000000	148.500000	2803.500000	15.500000	76.000000	1.000000
75%	29.000000	8.000000	262.000000	3608.000000	17.175000	79.000000	2.000000
max	46.600000	8.000000	455.000000	5140.000000	24.800000	82.000000	3.000000

[6]:
```
df.columns = df.columns.str.strip()

df = df.replace("?", np.nan)

df["horsepower"] = pd.to_numeric(df["horsepower"], errors="coerce")

numeric_columns = ["mpg", "cylinders", "displacement","horsepower", "weight", "acceleration", "model year"]

for column in numeric_columns:
    df[column] = pd.to_numeric(df[column], errors="coerce")

df = df.drop_duplicates()
df = df.dropna(subset=numeric_columns)

print("Dataset cleaned successfully!")
print("Remaining rows:", len(df))

display(df.head())
```

```
Dataset cleaned successfully!
Remaining rows: 392
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130.0	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150.0	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150.0	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140.0	3449	10.5	70	1	ford torino

7]:
```
features = ["cylinders","displacement","horsepower","weight","acceleration","model year"]

X = df[features]
y = df["mpg"]

print("Input features:")
print(features)

print("\nTarget variable: mpg")
print("Input shape:", X.shape)
print("Target shape:", y.shape)
```

```
Input features:
['cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model year']
```

```
Target variable: mpg
Input shape: (392, 6)
Target shape: (392,)
```

[8]:
```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20,random_state=42)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 313
Testing samples: 79
```

[9]:
```
model = LinearRegression()

model.fit(X_train, y_train)

print("Auto MPG Linear Regression model trained successfully!")
```

```
Auto MPG Linear Regression model trained successfully!
```

[10]:
```
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

```
print("Model Evaluation Results")

print("\nMAE:", round(mae, 3))
print("MSE:", round(mse, 3))
print("RMSE:", round(rmse, 3))
print("R2 Score:", round(r2, 3))
```

```
Model Evaluation Results

MAE: 2.504
MSE: 10.502
RMSE: 3.241
R2 Score: 0.794
```

11]:
```
coefficients = pd.DataFrame({"Feature": features,"Coefficient": model.coef_})

display(coefficients)

print("Intercept:", round(model.intercept_, 3))
```

	Feature	Coefficient
0	cylinders	-0.116173
1	displacement	0.001013
2	horsepower	-0.002276
3	weight	-0.006561
4	acceleration	0.061736
5	model year	0.760636

```
Intercept: -15.058
```

[12]:
```
sample_car = pd.DataFrame([{"cylinders": 4,"displacement": 140,"horsepower": 90,"weight": 2500,"acceleration": 15,"model year": 76}])

predicted_mpg = model.predict(sample_car)[0]

print("Sample Car Details:")
display(sample_car)

print("Predicted fuel efficiency:", round(predicted_mpg, 2), "MPG")
```

```
Sample Car Details:
```

	cylinders	displacement	horsepower	weight	acceleration	model year
0	4	140	90	2500	15	76

```
Predicted fuel efficiency: 26.75 MPG
```

[13]:
```
model_data = {"model": model,"features": features}

joblib.dump(model_data, "auto_mpg_model.pkl")

print("Auto MPG model saved successfully!")
```

```
Auto MPG model saved successfully!
```

[14]:
```
import gradio as gr

def predict_mpg(
    cylinders,
    displacement,
    horsepower,
    weight,
    acceleration,
    model_year
):
    input_data = pd.DataFrame([{
        "cylinders": cylinders,
        "displacement": displacement,
        "horsepower": horsepower,
        "weight": weight,
        "acceleration": acceleration,
        "model year": model_year
    }])

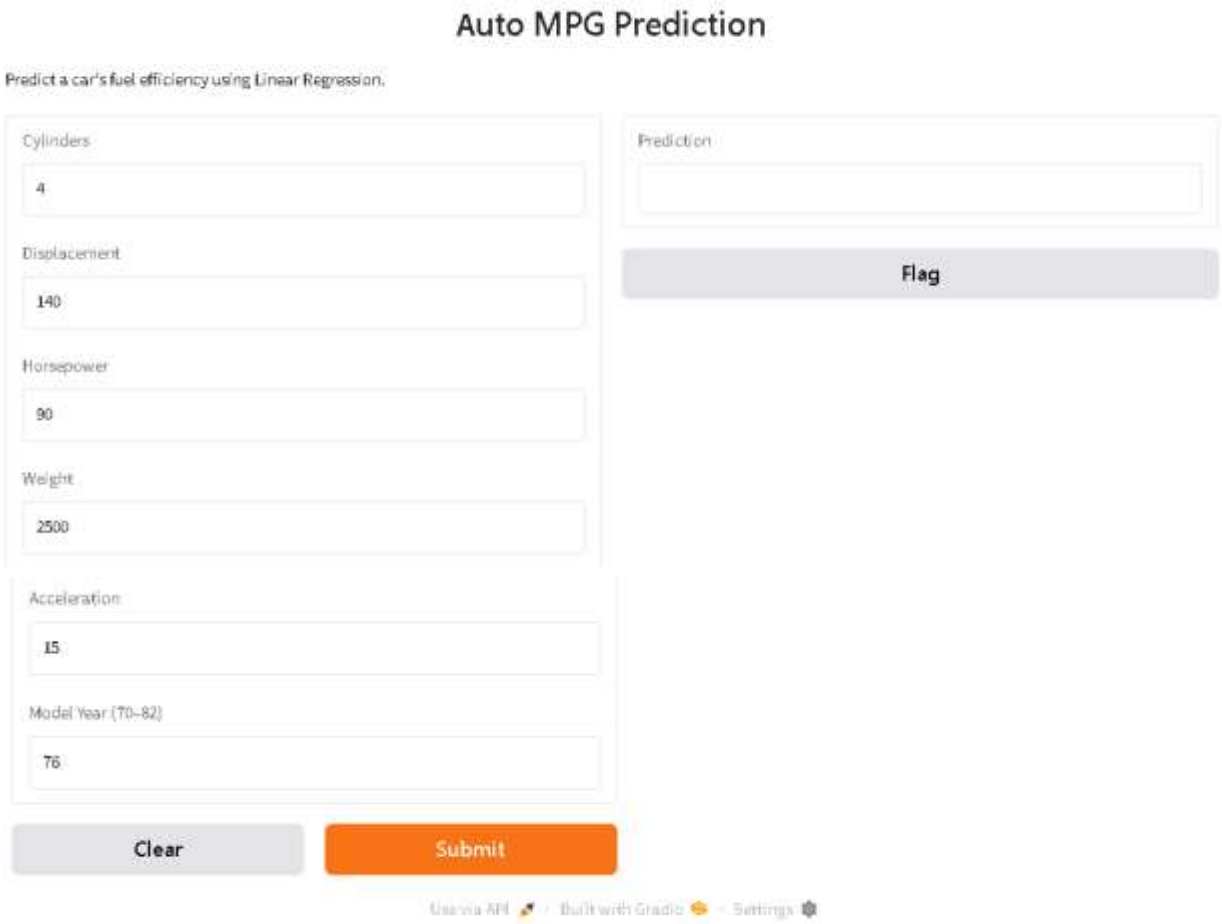
    prediction = model.predict(input_data)[0]

    return f"Predicted Fuel Efficiency: {prediction:.2f} MPG"

gradio_app = gr.Interface(
    fn=predict_mpg,
```

```
    inputs=[
        gr.Number(label="Cylinders", value=4),
        gr.Number(label="Displacement", value=140),
        gr.Number(label="Horsepower", value=90),
        gr.Number(label="Weight", value=2500),
        gr.Number(label="Acceleration", value=15),
        gr.Number(label="Model Year (70-82)", value=76)
    ],
    outputs=gr.Textbox(label="Prediction"),
    title="Auto MPG Prediction",
    description="Predict a car's fuel efficiency using Linear Regression."
)

gradio_app.launch()
```



[14]:

[15]:
```
os.makedirs("templates_auto", exist_ok=True)

print("Auto MPG template folder created!")
```

```
Auto MPG template folder created!
```

[16]:
```
html_code = """
<!DOCTYPE html>
<html>
<head>
    <title>Auto MPG Prediction</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f1f5f9;
            padding: 30px;
        }
        .container {
            max-width: 550px;
            margin: auto;
            background: white;
            padding: 25px;
            border-radius: 12px;
            box-shadow: 0 4px 12px #ccc;
        }
        h1 {
            color: #1d4ed8;
            text-align: center;
        }
        label {
            display: block;
            margin-top: 14px;
            padding: 10px;
            margin-top: 5px;
            box-sizing: border-box;
        }
        button {
            width: 100%;
            padding: 12px;
            margin-top: 20px;
            background: #2563eb;
            color: white;
            border: none;
            cursor: pointer;
        }
        .result {
            margin-top: 20px;
            padding: 15px;
            background: #dbeafe;
            border-radius: 6px;
            text-align: center;
```

```
        <label>Horsepower</label>
        <input type="number" name="horsepower"
               value="90" min="0" step="any" required>

        <label>Weight</label>
        <input type="number" name="weight"
               value="2500" min="0" step="any" required>

        <label>Acceleration</label>
        <input type="number" name="acceleration"
               value="15" min="0" step="any" required>

        <label>Model Year (70-82)</label>
        <input type="number" name="model year"
               value="76" min="70" max="82" required>

        <button type="submit">Predict MPG</button>
    </form>

    {% if prediction is not none %}
    <div class="result">
        <h3>Predicted Fuel Efficiency: {{ prediction }} MPG</h3>
    </div>
    {% endif %}
```

```
</body>
</html>
"""

with open("templates_auto/index.html", "w", encoding="utf-8") as file:
    file.write(html_code)

print("Auto MPG HTML page created successfully!")
```

```
Auto MPG HTML page created successfully!
```

17]:

```
from flask import Flask, render_template, request

# Load the Auto MPG model
auto_data = joblib.load("auto_mpg_model.pkl")

auto_model = auto_data["model"]
auto_features = auto_data["features"]

# Use a unique Flask app and a separate template folder
auto_app = Flask(
    "AutoMPGApp",
    template_folder="templates_auto"
)
```

```
@auto_app.route("/", methods=["GET", "POST"])
def auto_home():
    prediction = None
    error = None

    if request.method == "POST":
        try:
            values = [
                float(request.form[feature])
                for feature in auto_features
            ]

            input_df = pd.DataFrame(
                [values],
                columns=auto_features
            )

            result = auto_model.predict(input_df)[0]
            prediction = round(float(result), 2)

        except (ValueError, KeyError):
            error = "Please enter valid values in all fields."

    return render_template(
        "index.html",
        prediction=prediction,
        error=error
```

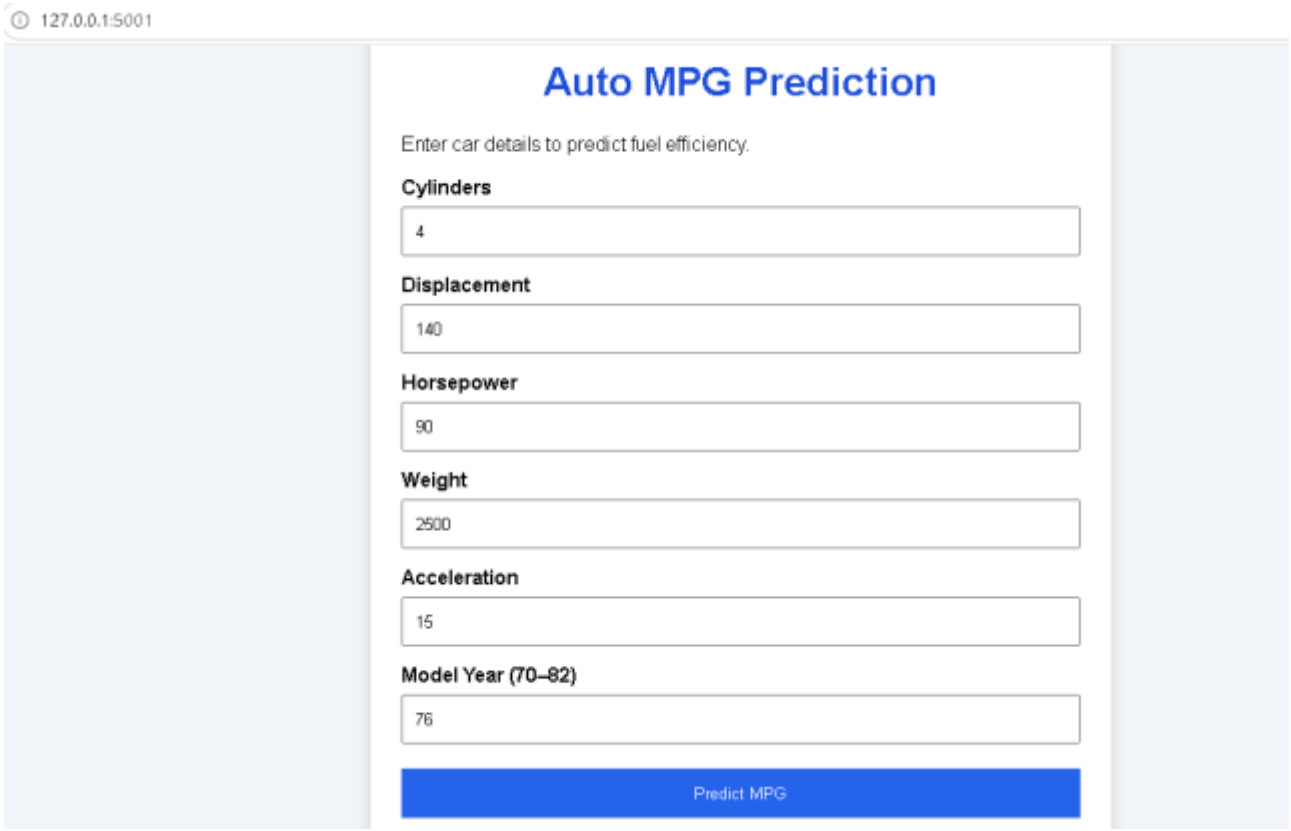
```
    return render_template(
        "index.html",
        prediction=prediction,
        error=error
    )

print("Auto MPG Flask application is ready!")
```

```
Auto MPG Flask application is ready!
```

```
[*]: auto_app.run(host="127.0.0.1",port=5001,debug=False,use_reloader=False)
```

```
 * Serving Flask app 'AutoMPGApp'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5001
Press CTRL+C to quit
```


127.0.0.1:5001
Auto MPG Prediction
Enter car details to predict fuel efficiency.
Cylinders
4
Displacement
140
Horsepower
90
Weight
2500
Acceleration
15
Model Year (70–82)
76
Predict MPG